

LLM Agents, LangChain & LangGraph

Detailed Q&A Revision Guide

Nityaa Kalra | August 2026

SECTION 1: WHAT ARE LLM AGENTS?

Q1: What is an LLM agent and how does it differ from a simple LLM call?

A simple LLM call: prompt in → text out. One step. No feedback loop.

An agent is an LLM in a loop: it can decide what to do next, use tools, observe results, and repeat — until it decides the task is complete. The LLM acts as the reasoning engine that plans and directs execution.

Components of an agent:

- **LLM**: the reasoning/planning engine
- **Tools**: functions the agent can call (search, calculator, SQL, code executor)
- **Memory**: what the agent remembers across steps or sessions
- **Orchestration loop**: the scaffolding that runs LLM → action → observation → repeat

Q2: What is the ReAct pattern?

ReAct (Reason + Act) is the most widely used agent prompting pattern. The agent interleaves reasoning and acting in a structured format:

```
Thought: I need to find the F1 score of Qwen3-1.7B on SemEval-2016. Action:
search_results_table(model='Qwen3-1.7B', dataset='SemEval-2016') Observation:
{'macro_f1': 0.623, 'favor_f1': 0.71, 'against_f1': 0.58, 'none_f1': 0.58} Thought: I
have the result. I can now answer. Action: finish(answer='Qwen3-1.7B achieves 62.3%
macro F1 on SemEval-2016')
```

Each Thought explains the plan, Action calls a tool, Observation is the tool's return. The loop continues until the agent calls 'finish'.

Q3: What is tool use / function calling in LLM agents?

Modern LLMs (GPT-4, Claude, Gemini) support native function calling: you define tools as JSON schemas, the LLM outputs a structured tool call (not free text), and your code executes it and returns

the result back to the LLM.

```
tools = [ { "name": "search_paper_results", "description": "Search stance detection experiment results", "parameters": { "type": "object", "properties": { "model_name": { "type": "string", "dataset": { "type": "string", "enum": ["SemEval-2016", "EZStance-NP"] } }, "required": ["model_name"] } } } ] # LLM responds with structured call: # { "name": "search_paper_results", # "arguments": { "model_name": "Qwen3-4B", "dataset": "SemEval-2016" }}
```

SECTION 2: LANGCHAIN

Q4: What is LangChain and what problems does it solve?

LangChain is a framework for building LLM applications. It provides abstractions for: connecting to LLMs, building chains of operations, adding memory, integrating tools and retrievers, and building agents.

Without LangChain: you manually manage prompt templates, API calls, parsing outputs, connecting retrieval results back to prompts, etc. LangChain standardizes these patterns.

```
from langchain_openai import ChatOpenAI from langchain_core.prompts import ChatPromptTemplate from langchain_core.output_parsers import StrOutputParser # Simple chain: prompt -> LLM -> parse llm = ChatOpenAI(model='gpt-4o', temperature=0) prompt = ChatPromptTemplate.from_messages([ ('system', 'You are a stance detection expert.'), ('human', 'Classify the stance in: {text}') ]) chain = prompt | llm | StrOutputParser() result = chain.invoke({'text': 'I strongly support climate legislation'}) print(result) # 'FAVOR'
```

Q5: What are the key LangChain components?

Component	What it does
ChatPromptTemplate	Structured prompt with variables
ChatOpenAI / ChatAnthropic	LLM connector with standard interface
StrOutputParser / JsonOutputParser	Parse LLM output to string/dict
ConversationBufferMemory	Stores chat history for multi-turn
VectorStoreRetriever	Retrieves relevant chunks from vector DB
LLMChain	Basic prompt + LLM chain
RetrievalQA	RAG: retriever + LLM for Q&A

Q6: How do you build a RAG chain in LangChain?

```
from langchain_community.vectorstores import Qdrant from langchain_openai import OpenAIEmbeddings, ChatOpenAI from langchain_core.prompts import ChatPromptTemplate from langchain_core.runnables import RunnablePassthrough # Set up retriever vectorstore = Qdrant(client=qdrant_client, collection_name='papers', embeddings=OpenAIEmbeddings()) retriever = vectorstore.as_retriever(search_kwargs={'k': 5}) # RAG prompt prompt = ChatPromptTemplate.from_template(""" Answer the question using only the context below. Context: {context} Question: {question} """) # Build RAG chain using LCEL (LangChain Expression Language) rag_chain = ( {'context': retriever, 'question': RunnablePassthrough()} | prompt | ChatOpenAI(model='gpt-4o') | StrOutputParser() )
```

```
result = rag_chain.invoke('What is the macro F1 of Llama on EZStance?')
```

SECTION 3: LANGGRAPH

Q7: What is LangGraph and how does it differ from LangChain?

LangChain: good for linear workflows — chain A flows into B flows into C. Simple sequential operations. No branching based on LLM decisions mid-flow.

LangGraph: extends LangChain with stateful, graph-based workflows. Nodes are functions (LLM calls, tool calls, logic), edges define flow between them. Supports cycles, conditional routing, human-in-the-loop, and persistent state.

Use LangGraph when:

- You need loops (agent retries a step if it fails)
- You need conditional branching (different path based on LLM output)
- You need human approval checkpoints mid-flow
- You're building multi-agent systems

Q8: What does a LangGraph agent look like in code?

```
from langgraph.graph import StateGraph, END from typing import TypedDict, Annotated, List import operator # Define state class AgentState(TypedDict): messages: Annotated[List, operator.add] next_action: str # Define nodes def call_llm(state: AgentState): response = llm.invoke(state['messages']) return {'messages': [response]} def call_tool(state: AgentState): last_msg = state['messages'][-1] result = execute_tool(last_msg.tool_calls[0]) return {'messages': [result]} def should_continue(state: AgentState): last = state['messages'][-1] if last.tool_calls: return 'call_tool' # route to tool execution return END # agent is done # Build graph graph = StateGraph(AgentState) graph.add_node('llm', call_llm) graph.add_node('tool', call_tool) graph.add_conditional_edges('llm', should_continue, {'call_tool': 'tool', END: END}) graph.add_edge('tool', 'llm') # after tool, back to LLM graph.set_entry_point('llm') app = graph.compile()
```

SECTION 4: MEMORY IN AGENTS

Q9: What are the types of memory in LLM agents?

Memory Type	What it stores	Scope
In-context (buffer)	Full conversation history in prompt	Single session
Summary memory	LLM-summarized history (saves tokens)	Single session
Entity memory	Facts about named entities mentioned	Single session
Vector store memory	Semantic search over past interactions	Cross-session
External DB	Structured facts in SQL/key-value store	Cross-session

Context window limit is the key constraint. Full history eventually overflows. Summary memory trades detail for token efficiency.

SECTION 5: AGENTIC SYSTEMS IN PRACTICE

Q10: What can go wrong with agents in production?

This is a real question for a senior role — they want you to think about failure modes.

Infinite loops: agent keeps calling tools without terminating. Fix: max iteration limit, timeout.

Tool errors: external API is down, returns unexpected format. Fix: error handling in tool functions, fallback tools.

Prompt injection: malicious content in retrieved documents tries to hijack agent. Fix: sandbox tool execution, validate outputs.

Cost spiraling: agent makes 50 LLM calls for a simple task. Fix: step limits, use smaller models for sub-tasks, caching.

Hallucinated tool calls: LLM calls a tool with wrong parameters or a tool that doesn't exist. Fix: structured output validation, explicit tool schemas.

Q11: How does vLLM fit into an agentic system?

In an agent loop, the LLM is called multiple times per task. If you use an open-source model (Qwen3, Llama), you need to serve it efficiently. That's where vLLM comes in.

vLLM serves open LLMs via an OpenAI-compatible API. Its key innovation is PagedAttention — manages KV cache in non-contiguous memory pages, allowing much higher throughput (10-24x) and concurrent request handling.

```
# Serve Qwen3-4B with vLLM vllm serve Qwen/Qwen3-4B --port 8000 # Use it in LangChain
exactly like OpenAI from langchain_openai import ChatOpenAI llm = ChatOpenAI(
base_url='http://localhost:8000/v1', api_key='not-needed', model='Qwen/Qwen3-4B' ) #
Now use in any LangChain chain or LangGraph agent result = llm.invoke('Classify the
stance: I support renewable energy')
```

This is directly how your project works: vLLM serves the SLMs, LangChain (or your custom code) wraps them for evaluation. Mention this explicitly.

Q12: What is the difference between an agent and a multi-agent system?

Single agent: one LLM orchestrator, many tools. Works for most tasks.

Multi-agent: multiple specialized LLM agents that collaborate. One agent might plan, another retrieves, another writes, another validates. LangGraph supports multi-agent orchestration natively.

When to use multi-agent: tasks too complex for one context window, parallel execution needed, specialized expertise (different system prompts for different sub-tasks).